

集合通信库

Collective Communication Library

时 间：2025.08.16

职 位：智源AI框架研究员

— 目录

- 01 基本概念
- 02 常见集合通信操作
- 03 常见AI通信范式和优化方法
- 04 NCCL&FlagCX工作流

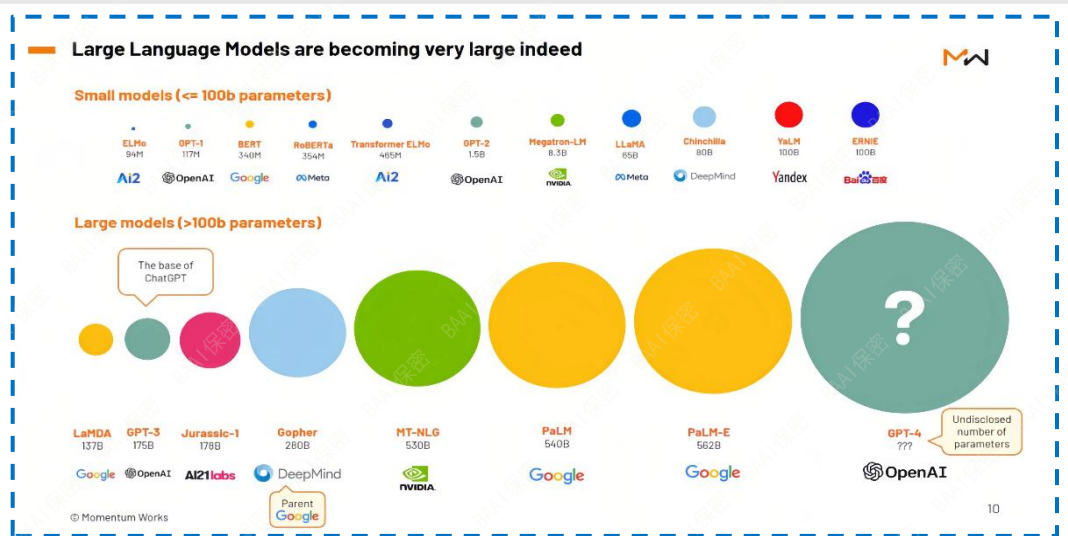
通信库

一个为分布式并行计算系统实现交互的软件库，可实现多种物理设备通过不同传输协议进行高效通信以及与计算负载之间高效协调的功能，包括：

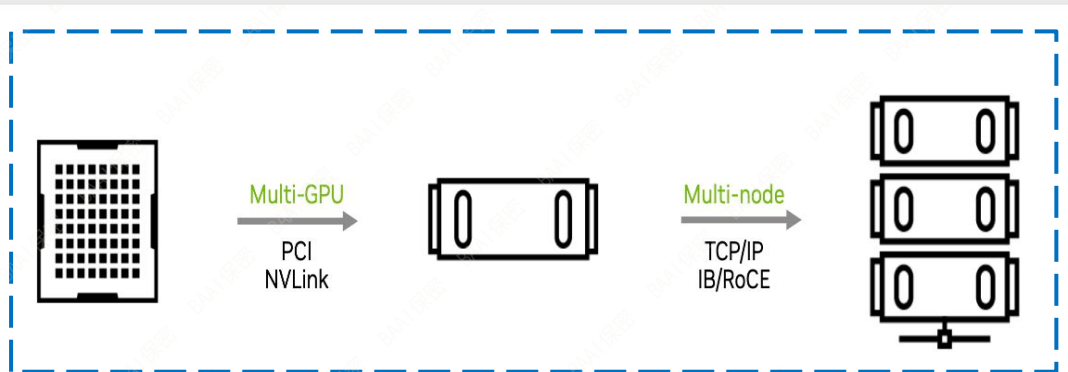
- 1、支持大规模分布式系统的高效通信
- 2、支持不同通信协议，包括Message Passing、PGAS等
- 3、支持多种通信操作，包括点对点通信、集合通信
- 4、支持不同通信传输协议，包括节点内（PCIe、NVLink等）和节点间（TCP/IP、IB/RoCE等）

应用场景

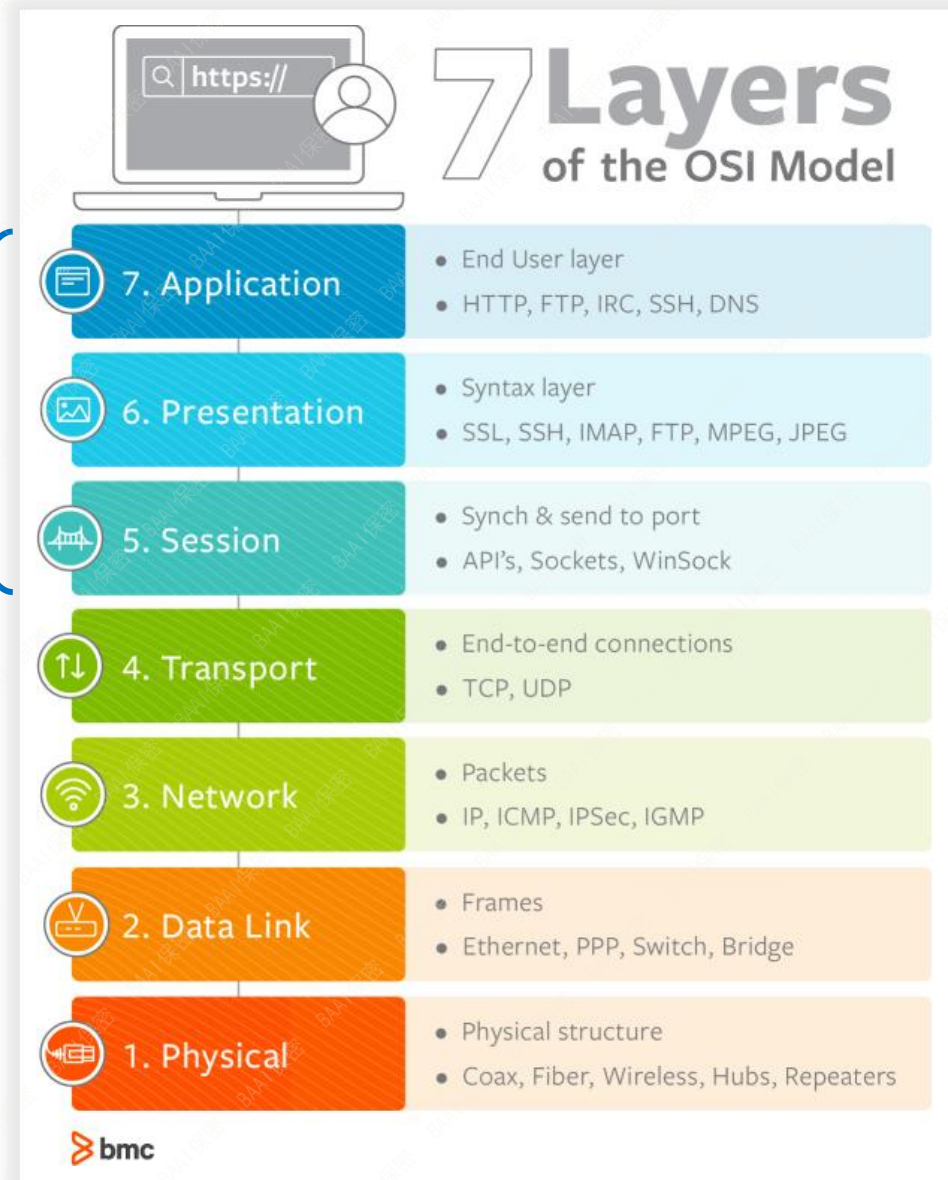
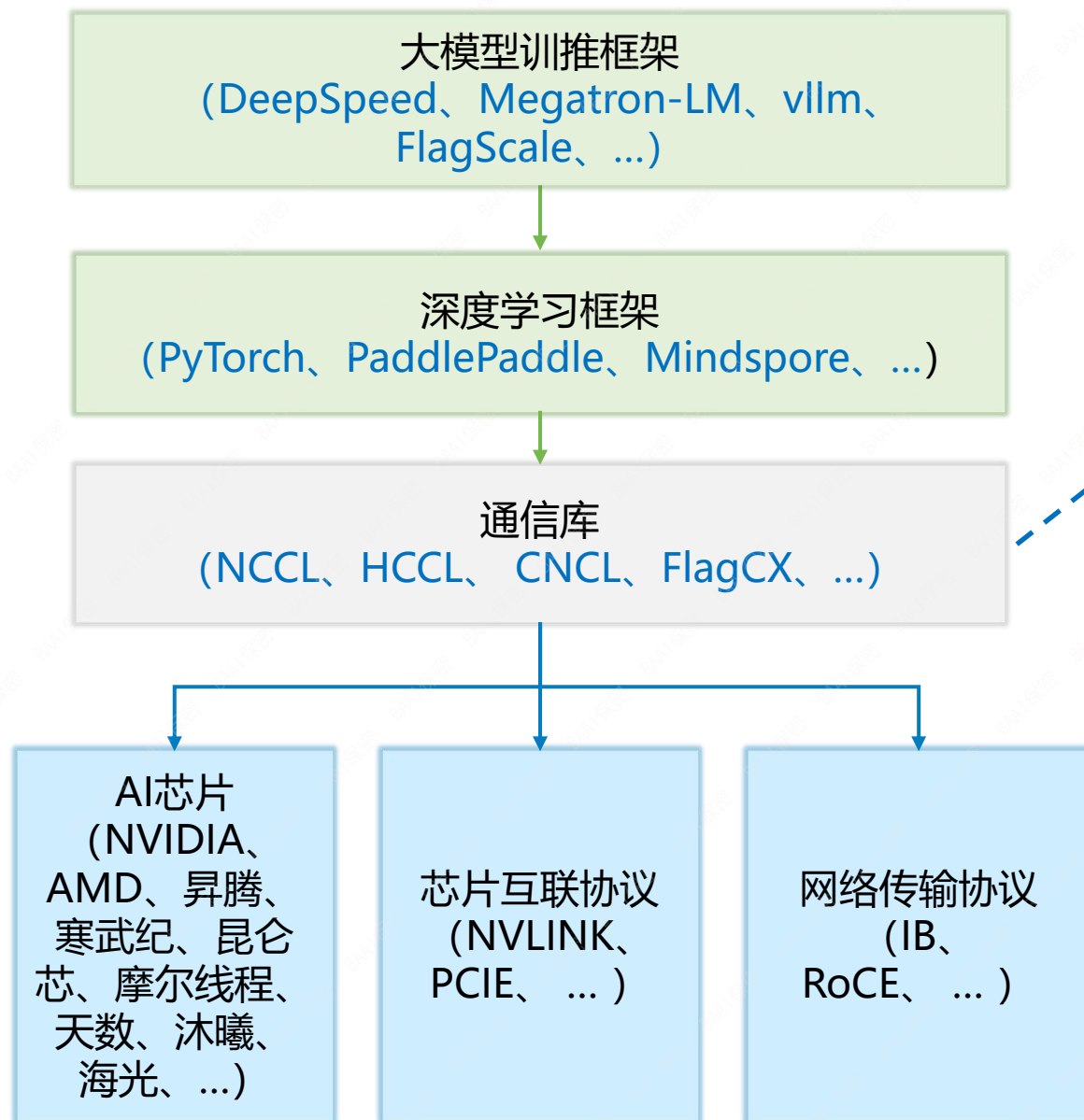
AI（千卡及万卡规模训练）、HPC



<https://www.bureauworks.com/blog/what-is-large-language-models-llm>



<https://www.nvidia.com/en-us/on-demand/session/gtc24-s62129/>

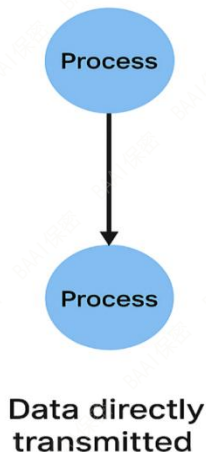


通信库	厂商	开源	基于其他通信库	异构通信	通信算法	传输协议	可观测性	拓扑感知
NCCL	NVIDIA	Y	N	N	Tree, Ring	PCIe、NVLink、SHM、IB/RoCE	nccl_test、nsys、debug flags	服务器内拓扑
RCCL	AMD	Y	N (NCCL API)	N	Tree, Ring	PCIe、Infinity Fabric、SHM、RoCE	rccl_test、roc_prof(tracing、可视化)、debug flags	HIP runtime, ROCm
oneCCL	Intel	Y	N/支持NCCL	N	Tree, Ring, Mesh, Hybrid	AVX-512、Omni-Path、oneAPI生态	vtune、trance analyzer、debug flags	Omni-path oneAPI
MSCCL	Microsoft	Y	NCCL	N	Tree, Ring, Customize	同NCCL	nccl_test、NPKit	任意拓扑的算法模拟
HCCL	Huawei	Y	N	N	Mesh, Ring, RHD, PairWise	HCCS、RoCE、PCIe	hccl_test	服务器内、服务器间拓扑
BCCL	Baidu	N	NCCL	N	同NCCL	同NCCL	bccl_test、支持通信带宽实时统计	同NCCL
ACCL	Alibaba Cloud	N	NCCL	N	同NCCL	同NCCL	accl_test、提供集合通信统计分析能力	NCCL基础上提高网络拓扑感知能力
TCCL	Tencent	N	NCCL	N	同NCCL	同NCCL	tccl_test	NCCL基础上提高网络拓扑感知能力
FlagCX	BAAI	Y	x-CCL	Y	C2C	IB/RoCE, TCP/IP	perf/test	服务器内拓扑

Point-to-Point Communication (点对点通信)

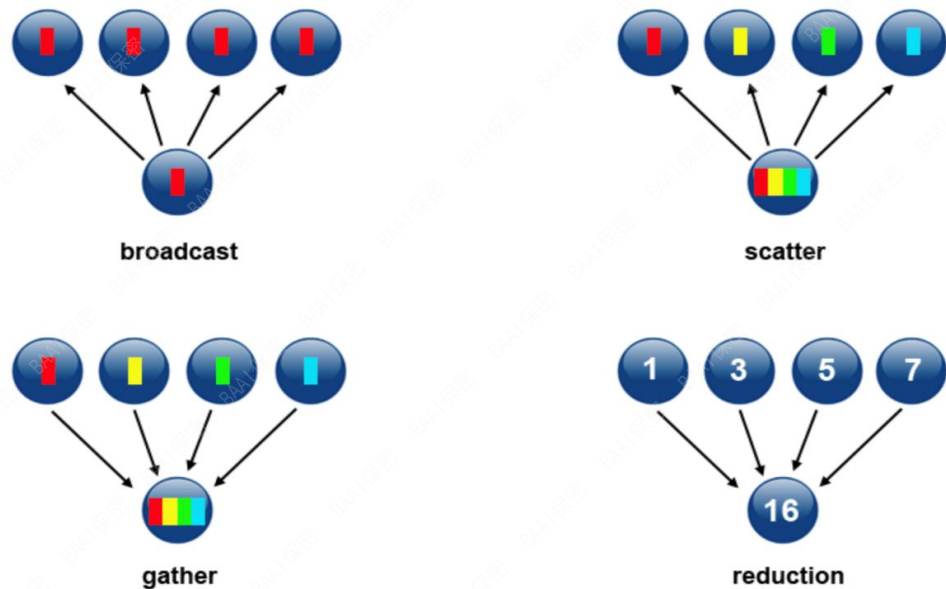
- ❑ 数据在两个进程间直接传输
- ❑ 一个进程作为发送方，另一个作为接收方
- ❑ 常见例子：
 - MPI中的MPI_Send/MPI_Recv
 - NCCL中的ncclSend/ncclRecv

Point-to-Point Communication



Collective Communication (集合通信)

- ❑ 一组进程共同参与的通信操作
- ❑ 数据可能会在组内进行分发、聚合、规约或广播
- ❑ 常见例子：
 - MPI中的MPI_Bcast/MPI_Allreduce
 - NCCL中的ncclBroadcast/ncclAllReduce



三类集合通信算法

一、规约 – 组内成员从其他成员收集数据，并对这些数据执行规约计算 (Min、Max、Sum、Prod、Avg等)

- ① AllReduce
- ② ReduceScatter
- ③ Reduce

二、数据传输 – 组内成员仅进行数据传输

- ① 广播 (Broadcast)
- ② 分发/收集 (Scatter、Gather、AllGather)
- ③ 全互联 (AlltoAll)

三、同步 - 进程会等待，直到组内所有成员到达同步点

- ① Barrier

三类集合通信算法

一、规约 – 组内成员从其他成员收集数据，并对这些数据执行规约计算（Min、Max、Sum、Prod、Avg等）

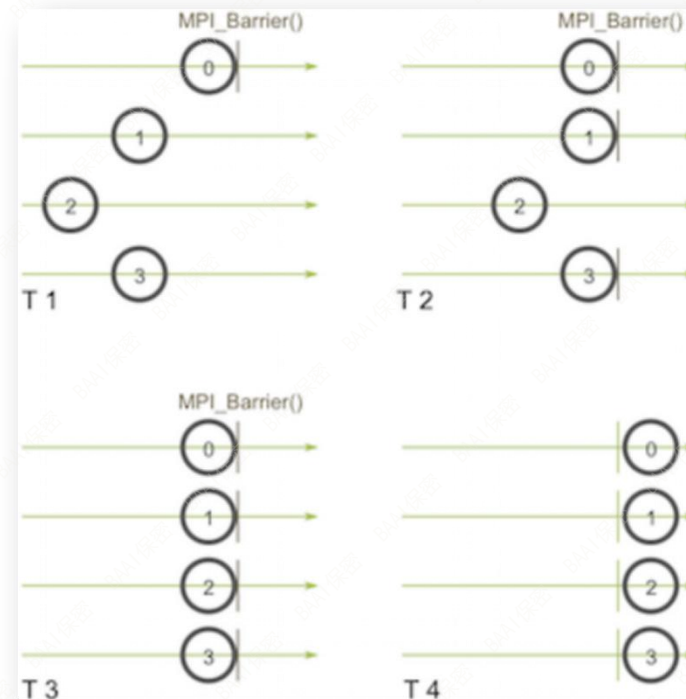
- ① AllReduce
- ② ReduceScatter
- ③ Reduce

二、数据传输 – 组内成员仅进行数据传输

- ① 广播 (Broadcast)
- ② 分发/收集 (Scatter、Gather、AllGather)
- ③ 全互联 (AlltoAll)

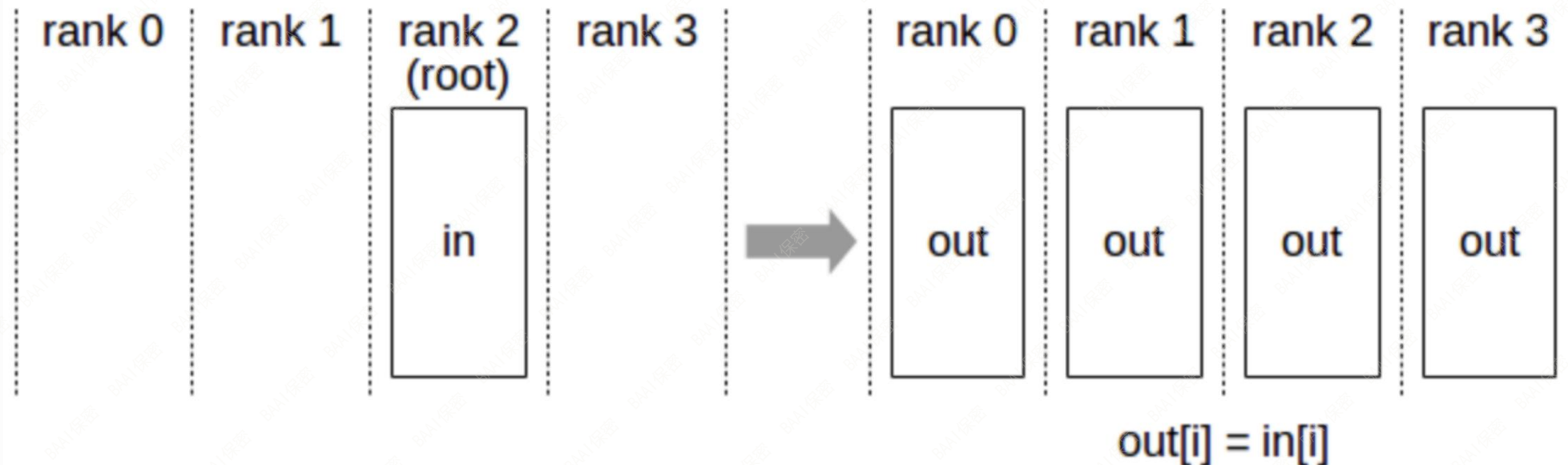
三、同步 - 进程会等待，直到组内所有成员到达同步点

- ① Barrier



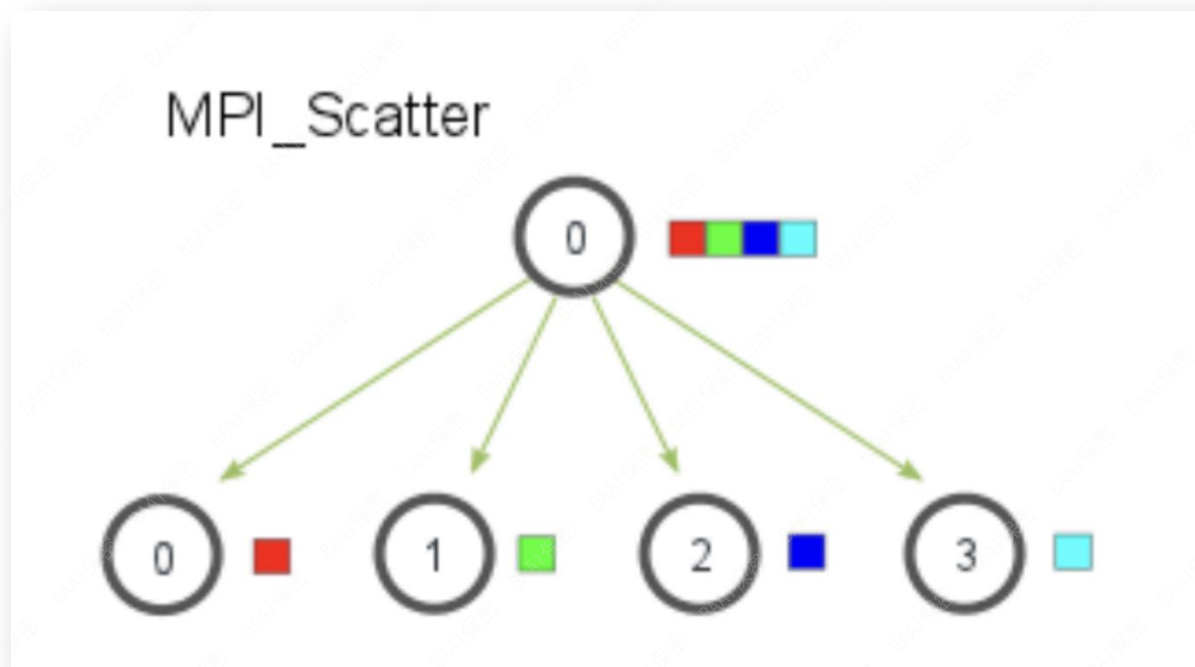
<https://mpitutorial.com/tutorials/mpi-broadcast-and-collective-communication/>

Broadcast: copies an N-element buffer from the root rank to all the ranks

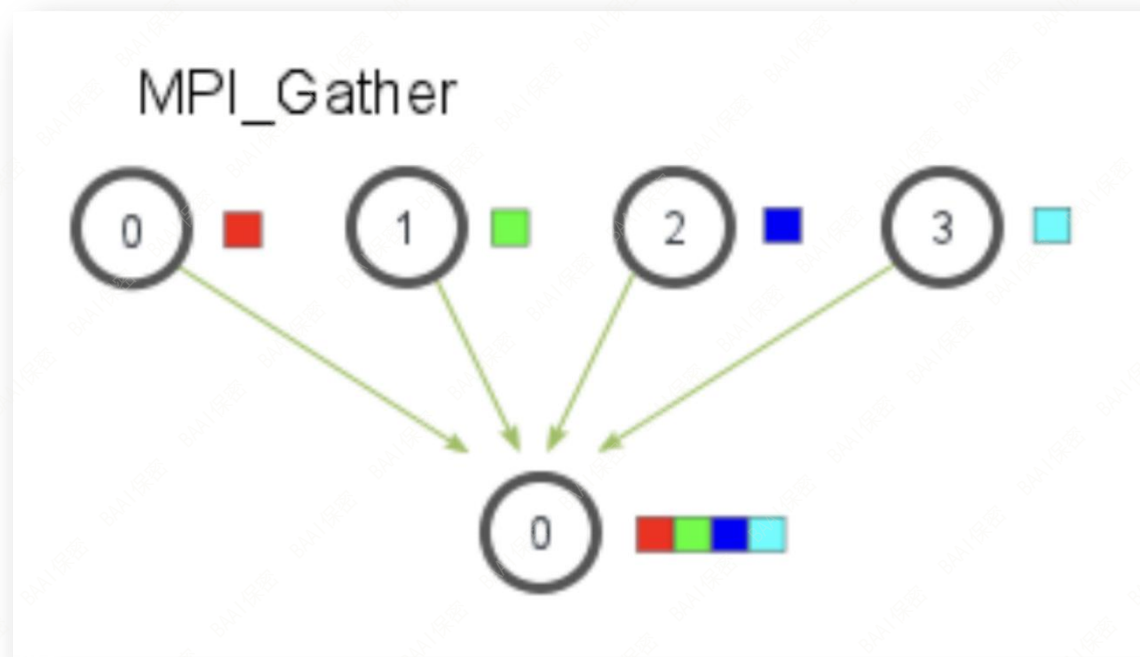


Broadcast operation: all ranks receive data from a “root” rank.

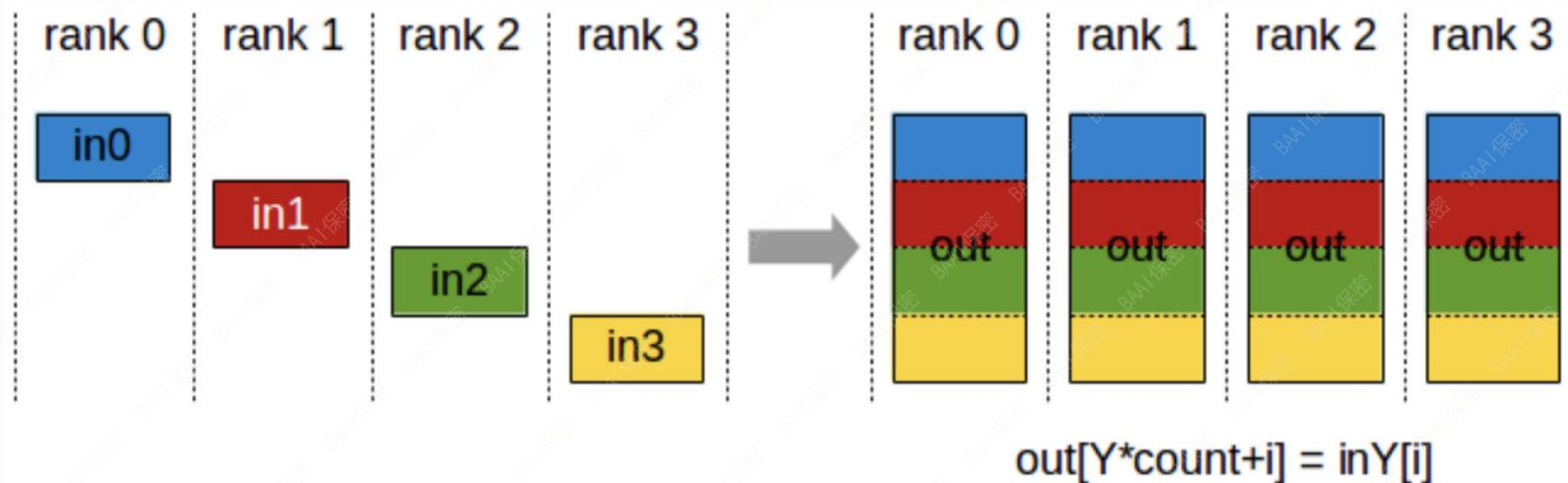
Scatter: distributes segments of an N-element buffer from the root rank to all the ranks, with each rank receiving exactly one distinct segment of the buffer



Gather: collects segments from all ranks and assembles them in a single N-element buffer at the root rank, with each rank contributing exactly one segment

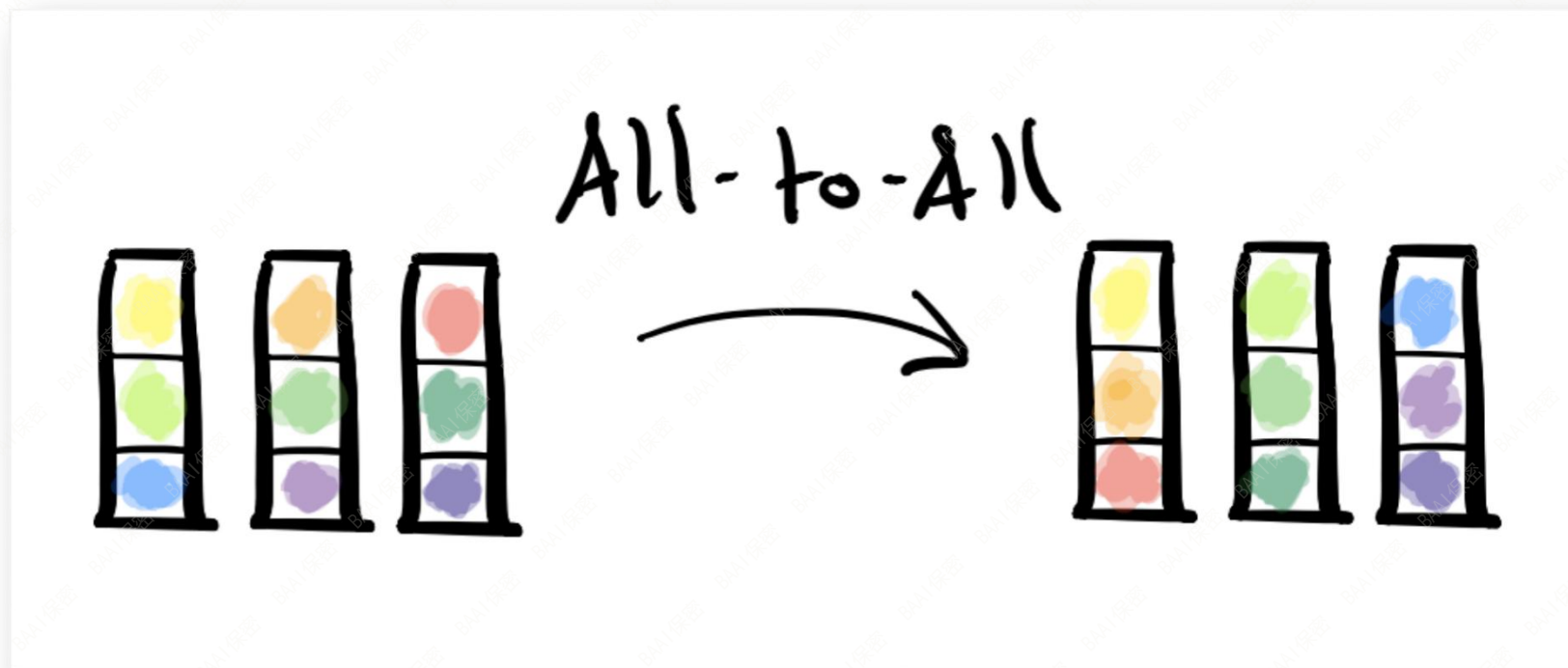


AllGather: gathers N values from k ranks into an output buffer of size $k*N$, and distributes that result to all ranks

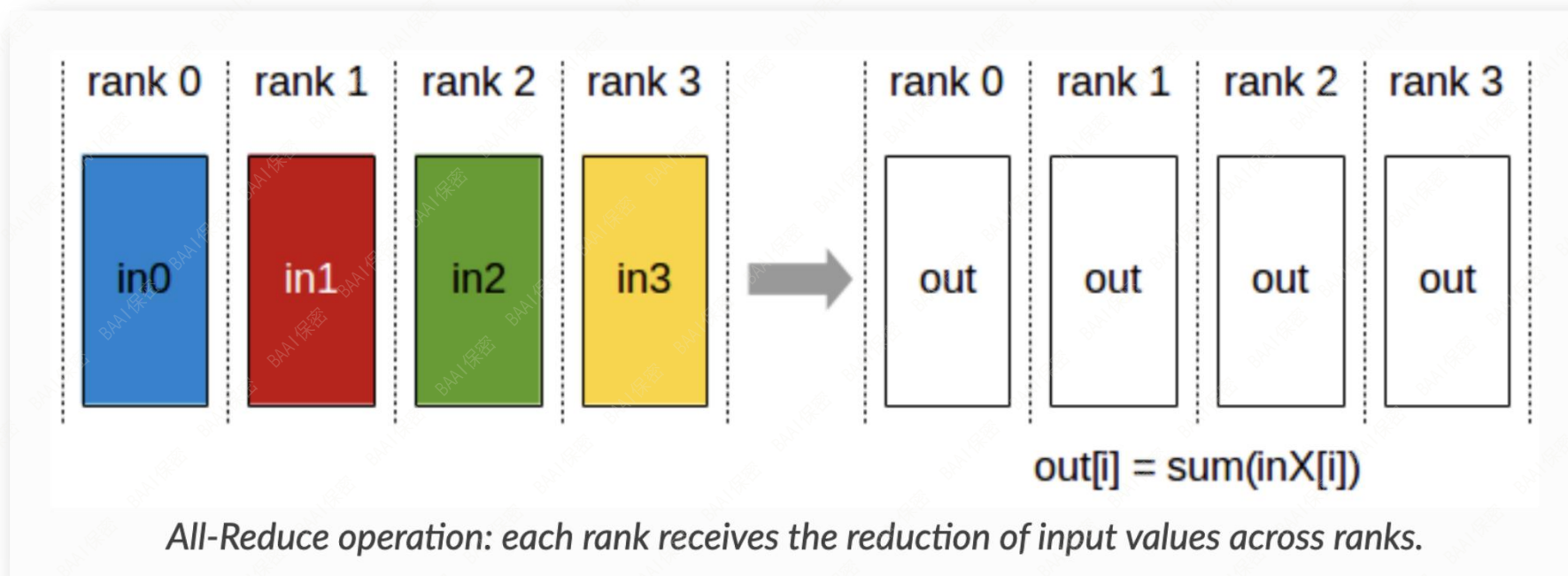


AllGather operation: each rank receives the aggregation of data from all ranks in the order of the ranks.

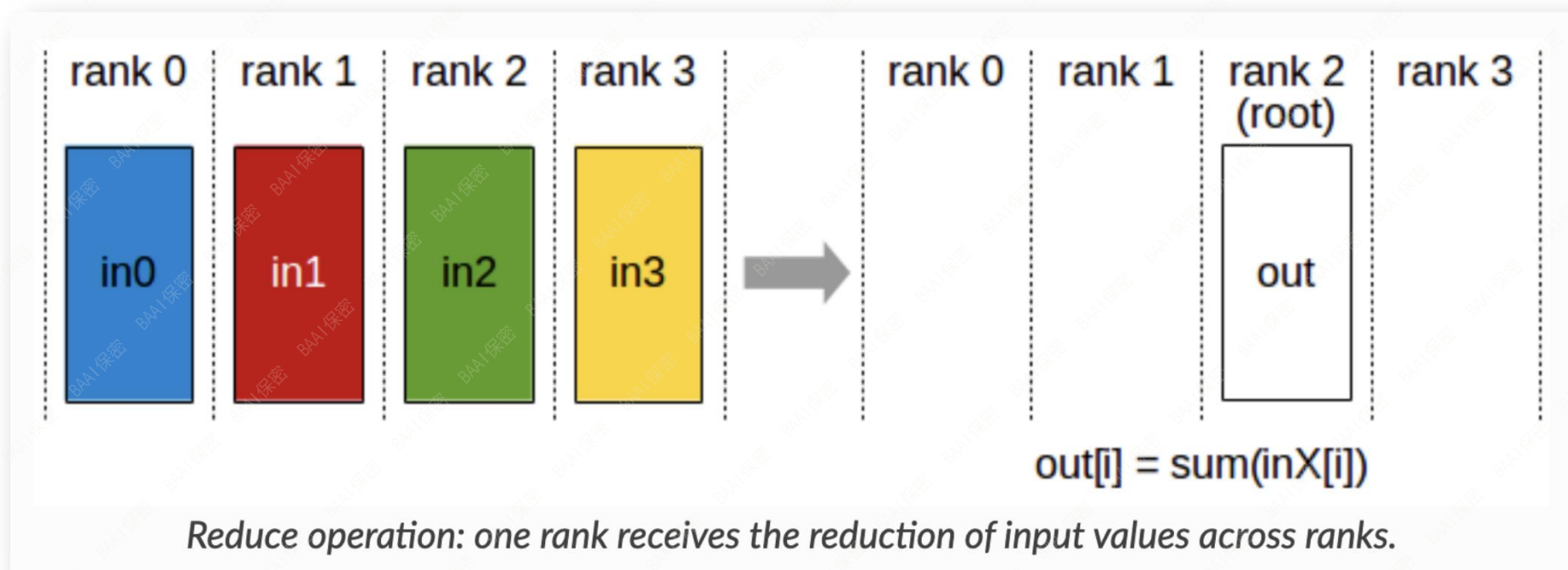
AlltoAll: each rank sends a distinct chunk of data to every other rank, and simultaneously receives a distinct chunk from every other rank.



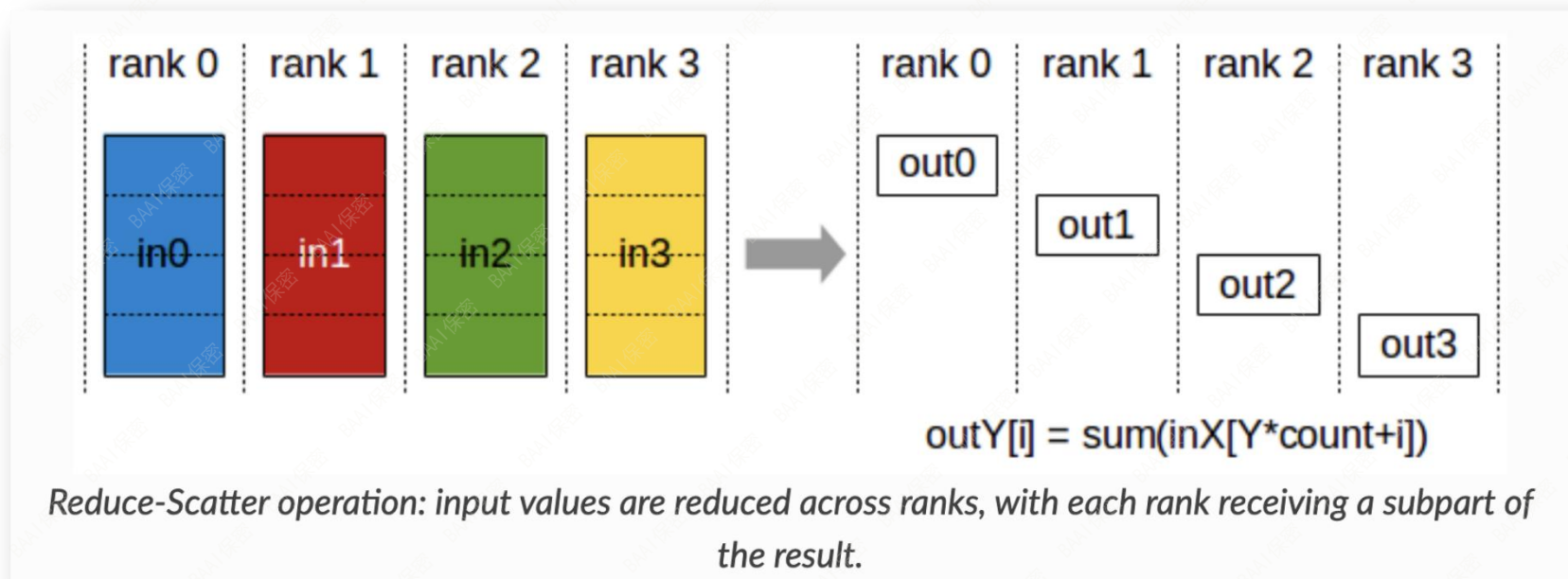
AllReduce: performs reductions on data (for example, sum, min, max) across ranks and stores the result in the receive buffer of every rank



Reduce: performs the same operation as AllReduce, but stores the result only in the receive buffer of a specified root rank



ReduceScatter : performs the same operation as Reduce, except that the result is scattered in equal-sized blocks between ranks, each rank getting a chunk of data based on its rank index

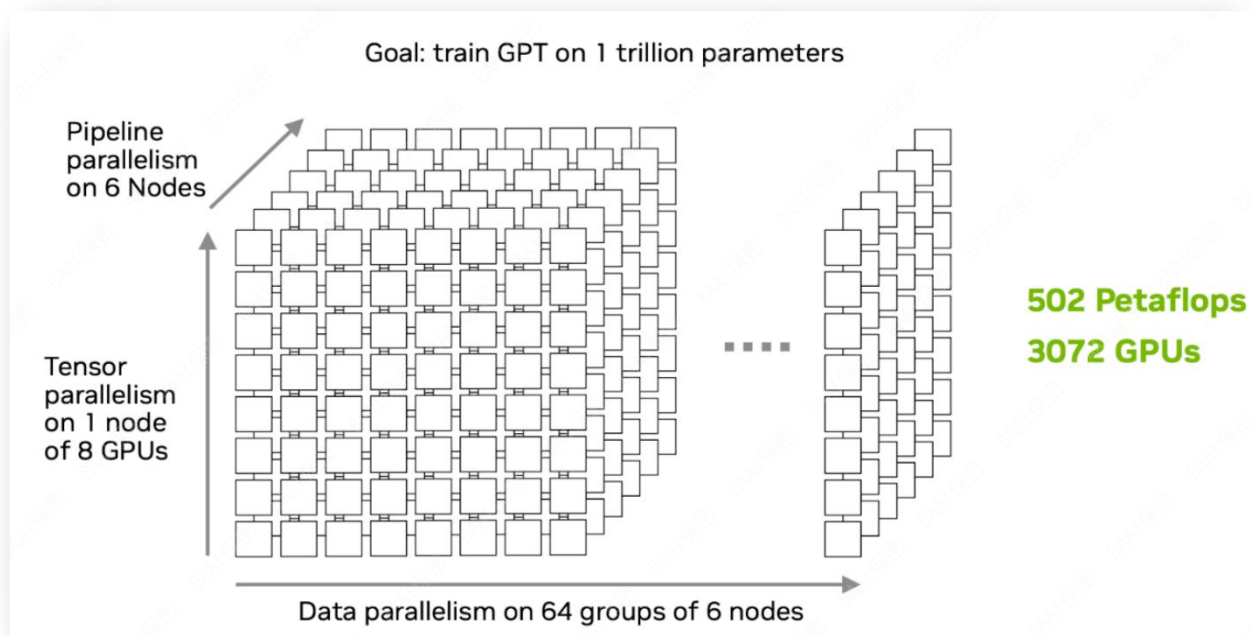


训练

- ① DP (Data parallelism)
- ② PP (Pipeline parallelism)
- ③ TP (Tensor parallelism)
- ④ CP (Context parallelism)
- ⑤ EP (Expert parallelism)
- ...

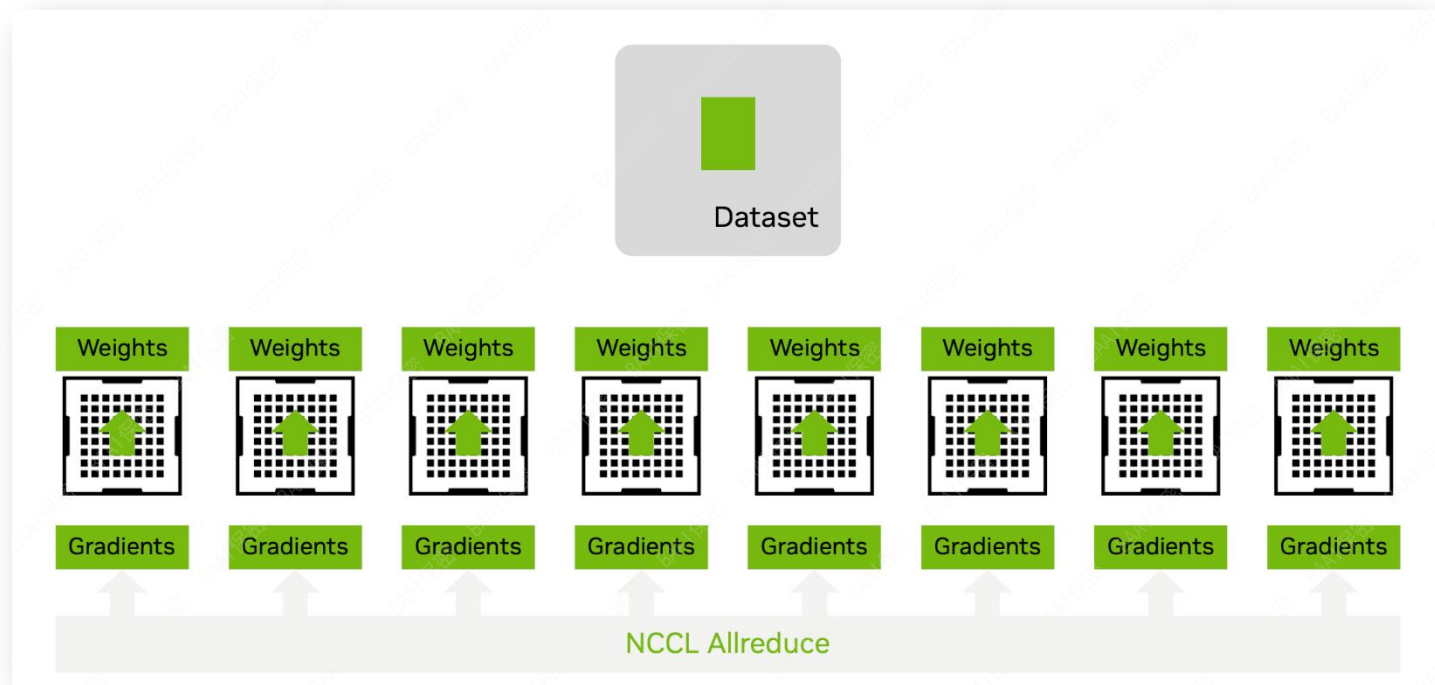
推理

- ① PDD (Prefilling Decoding Disaggregation)
- ② AFD (Attention-FFN Disaggregation)
- ...



常见AI通信范式和优化方法 – DP+ZeRO+Overlap

◆DP: Forward Pass + Backward Pass + AR + Parameter Update



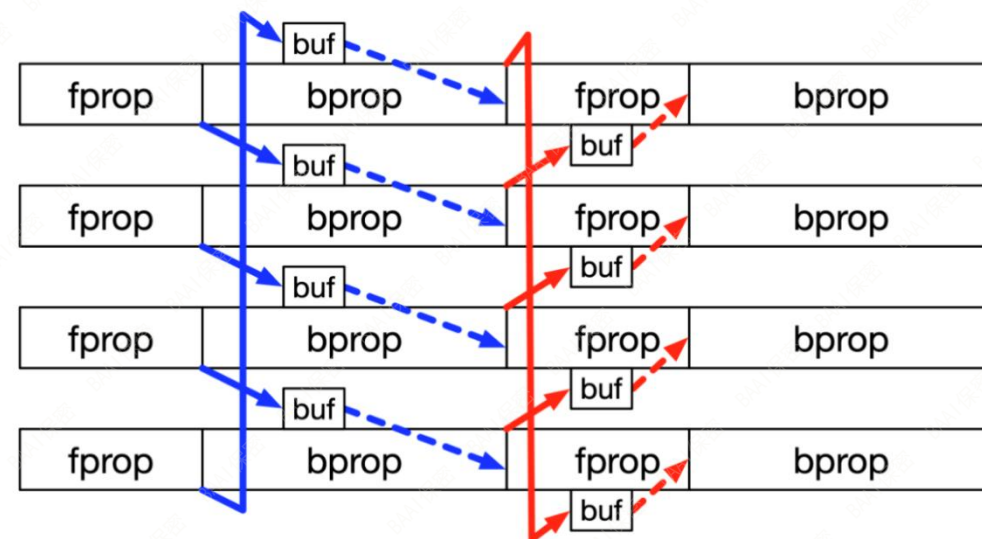
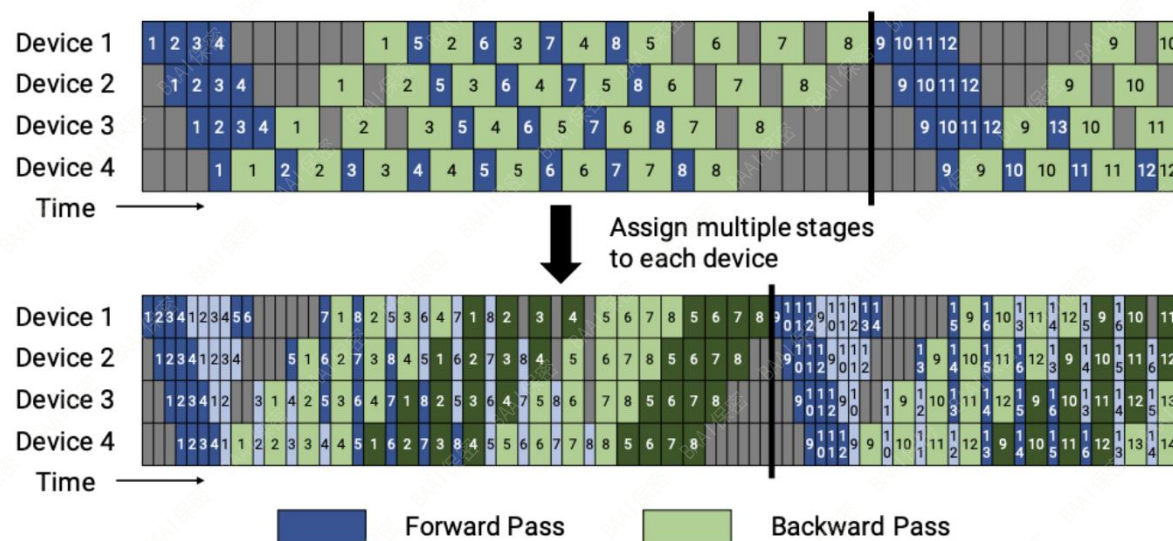
<https://www.nvidia.com/en-us/on-demand/session/gtc24-s62129/>

◆DP+ZeRO: Forward Pass + Backward Pass + RS + Parameter Update + AG

◆DP+ZeRO+Overlap: (AG+FP) + (BP+RS) + Parameter Update

常见AI通信范式和优化方法 – PP+Overlap

1F1B PP: Forward Pass + **SendRecv** + Backward Pass + **SendRecv** + Parameter Update



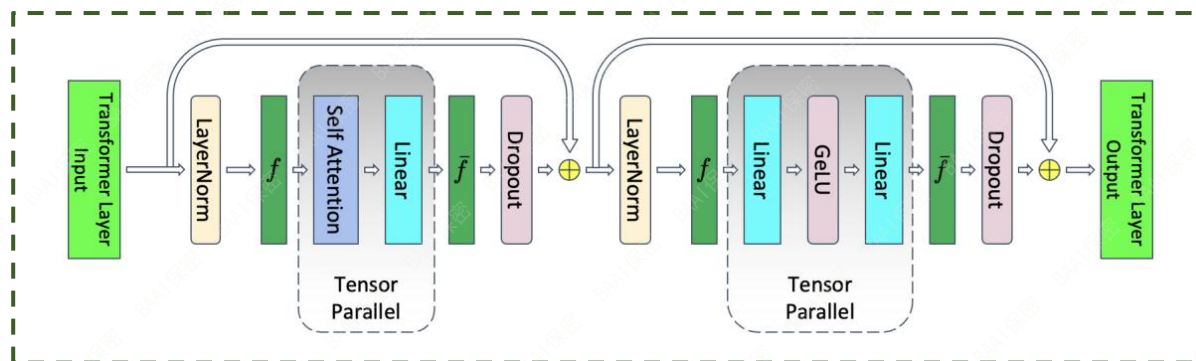
https://docs.nvidia.com/nemo-framework/user-guide/latest/nemotoolkit/features/optimizations/communication_overlap.html#pipeline-parallel-communication-overlap

[Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM](#)

PP+Overlap: **(FP+SendRecv)** + **(BP+SendRecv)** + Parameter Update

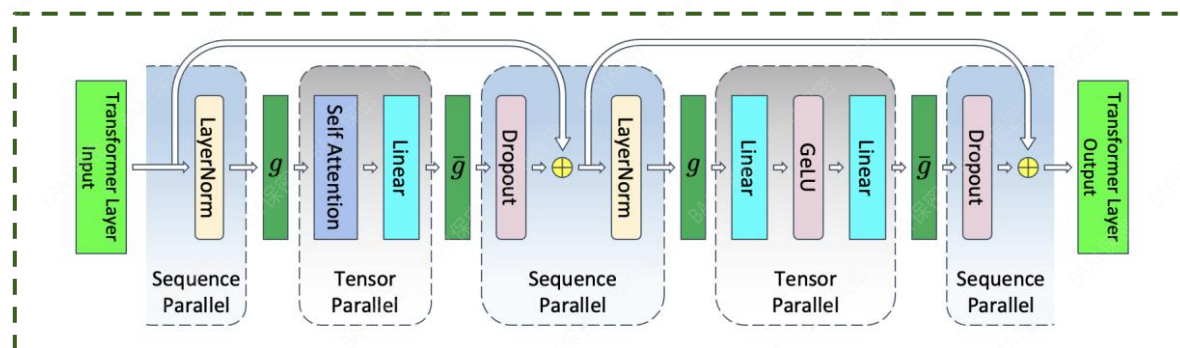
常见AI通信范式和优化方法 – TP+SP+Overlap

TP: FP (Attn + AR + MLP + AR) + BP (MLP + AR + Attn + AR) + PU

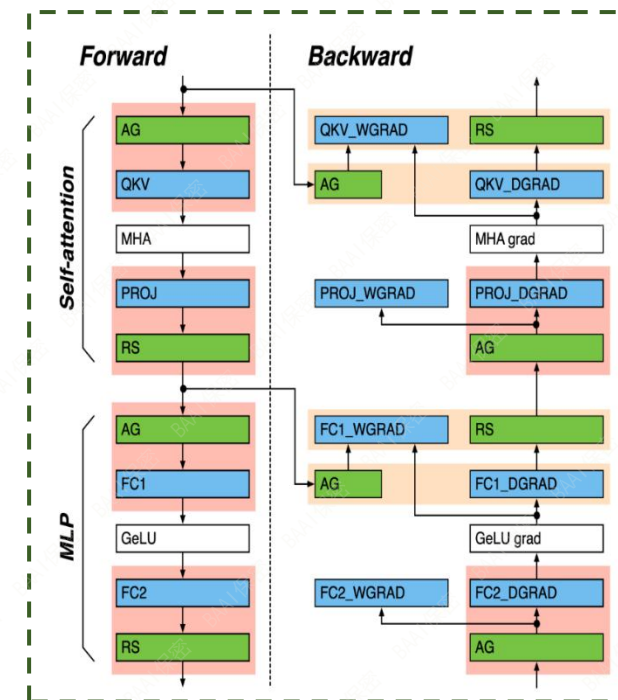


Reducing Activation Recomputation in Large Transformer Models

TP+SP: FP (AG + Attn + RS + AG + MLP + RS)
+ BP (2*AG + MLP + RS + 2*AG + Attn + RS)



Reducing Activation Recomputation in Large Transformer Models



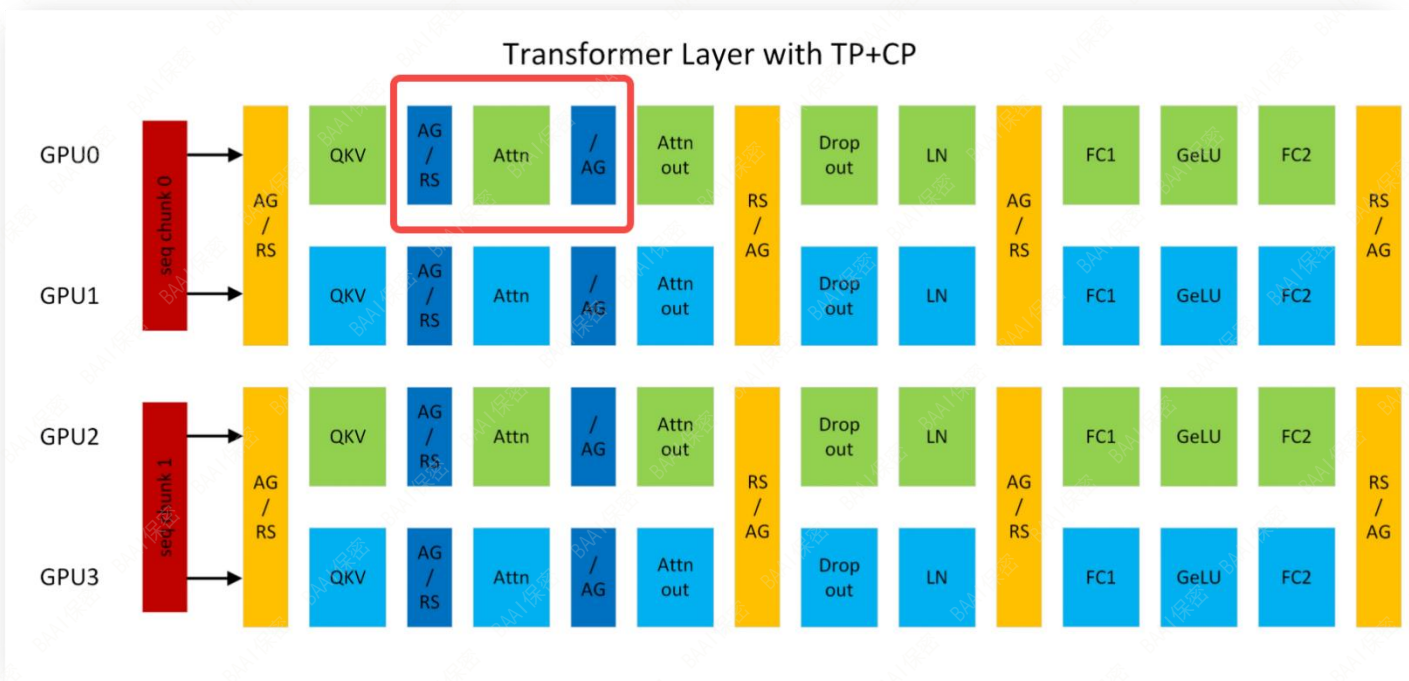
https://docs.nvidia.com/nemo-framework/user-guide/latest/nemotoolkit/features/optimizations/communication_overlap.html#tensor-parallel-communication-overlap

TP+SP+Overlap: FP (no-overlap)

+ BP (AG + (AG+MLP+RS) + AG + (AG+Attn+RS))

常见AI通信范式和优化方法 – CP+Overlap

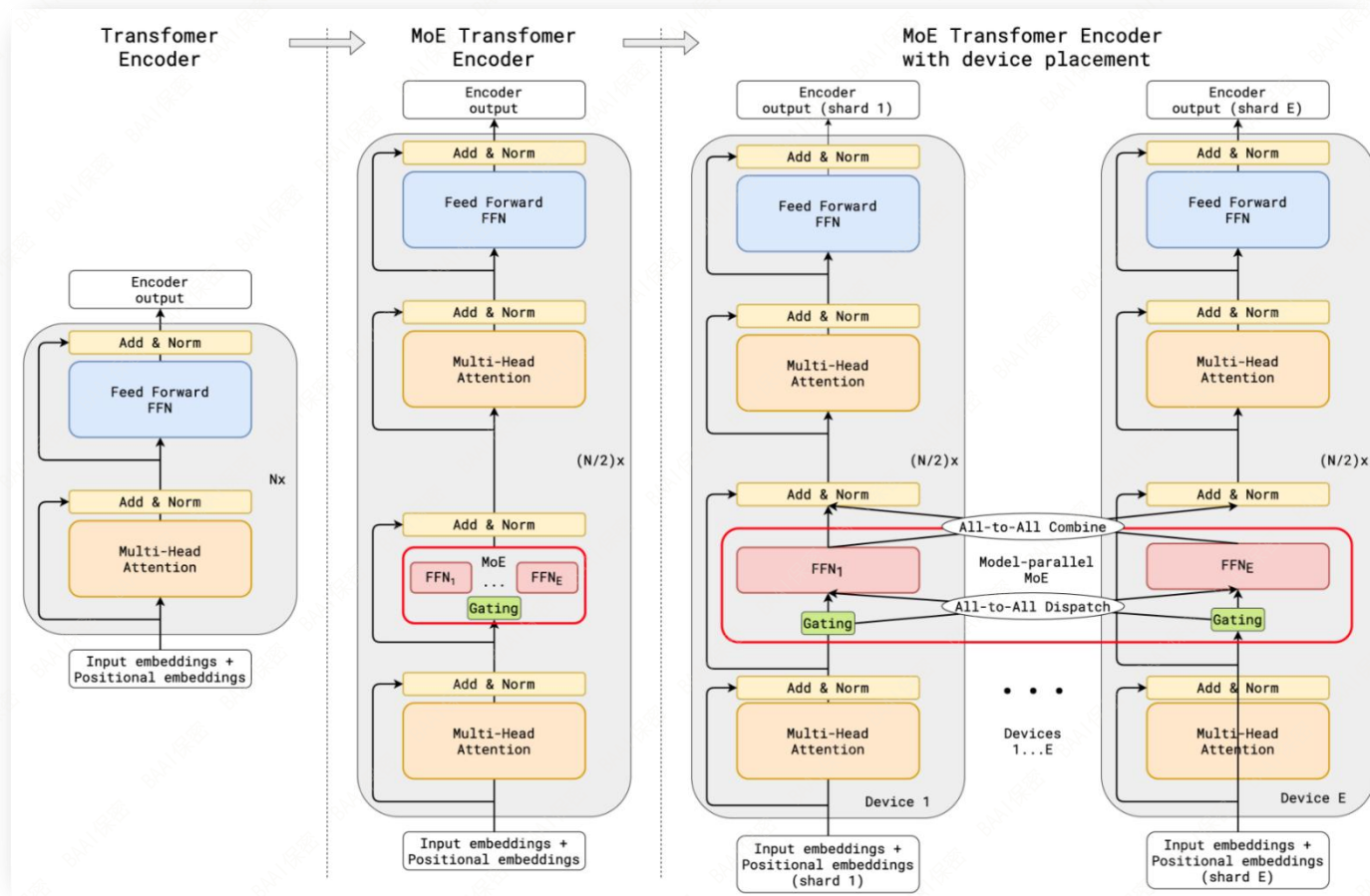
CP: FP (AG + Attn + MLP) + BP (MLP + AG + Attn + RS) + PU



https://docs.nvidia.com/megatron-core/developer-guide/latest/api-guide/context_parallel.html

CP+Overlap: FP (Attn+SendRecv + MLP) + BP (MLP + Attn+SendRecv) + PU

EP: FP (Attn + AlltoAll + MLP + AlltoAll) + BP (AlltoAll + MLP + AlltoAll + Attn) + PU

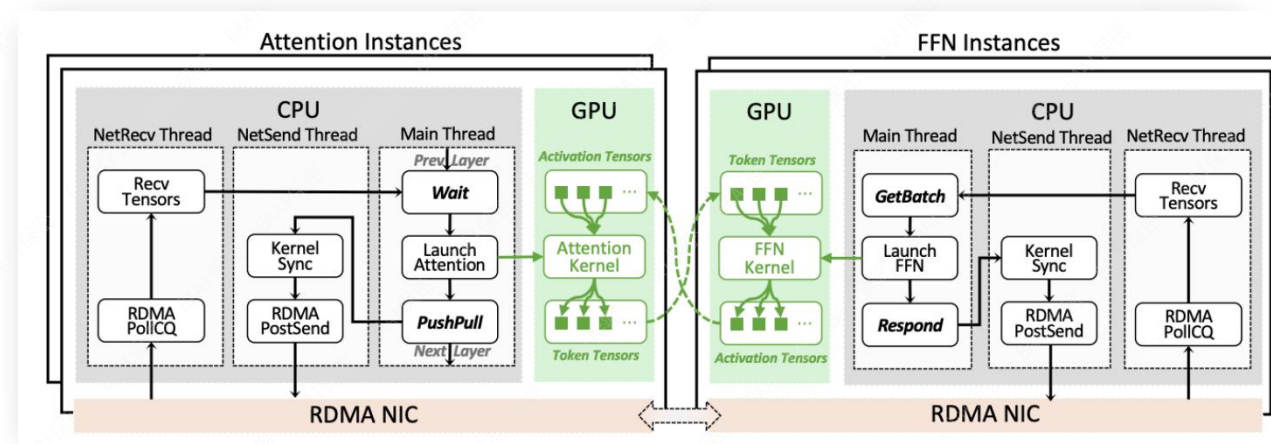
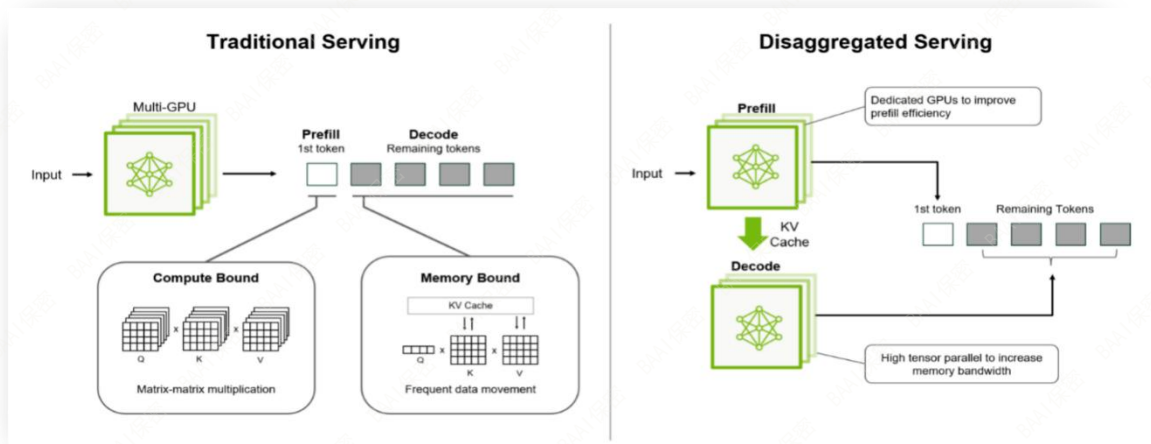


GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding

常见AI通信范式和优化方法 – PDD/AFD

PDD: prefilling + **SendRecv** + decoding

AFD: Attn + **SendRecv** + FFN



<https://developer.nvidia.com/blog/introducing-nvidia-dynamo-a-low-latency-distributed-inference-framework-for-scaling-reasoning-ai-models/>

Step-3 is Large yet Affordable: Model-system Co-design for Cost-effective Decoding

训练

- ① DP (Data parallelism)
- ② PP (Pipeline parallelism)
- ③ TP (Tensor parallelism)
- ④ CP (Context parallelism)
- ⑤ EP (Expert parallelism)
- ...

推理

- ① PDD (Prefilling Decoding Disaggregation)
- ② AFD (Attention-FFN Disaggregation)
- ...



通信范式

- ① AllReduce/ReduceScatter/AllGather: **DP, TP, CP**
- ② AlltoAll/SendRecv: **PP, CP, EP, PDD, AFD**

优化方法

- ① 通信隐藏: 调度隐藏, 通算融合
- ② 通信算法: 分层AR/A2A等
- ③ 通信压缩: ZeRo++等
- ④ 网络优化: PXN, P2P Engine等

NCCL workflow

- UniqueId&Bootstrap
- Topo&Graph
- CommOp
- Algo&Proto

FlagCX workflow

- UniqueId&Bootstrap
- Homo/HeteroComm
- CommOp

NCCL API

```
// Initialization, finalize, abort and fault tolerance
ncclResult_t ncclGetUniqueId(ncclUniqueId* uniqueId);
ncclResult_t ncclCommInitRank(ncclComm_t* comm, int nranks, ncclUniqueId commId, int rank);
ncclResult_t ncclCommInitRankConfig(ncclComm_t* comm, int nranks, ncclUniqueId commId, int rank, ncclConfig_t* config);
ncclResult_t ncclCommFinalize(ncclComm_t comm);
ncclResult_t ncclCommDestroy(ncclComm_t comm);
ncclResult_t ncclCommAbort(ncclComm_t comm);
ncclResult_t ncclCommGetAsyncError(ncclComm_t comm, ncclResult_t *asyncError);
ncclResult_t ncclCommSplit(ncclComm_t comm, int color, int key, ncclComm_t *newcomm, ncclConfig_t* config);

// Collective communication - since 2.0
ncclResult_t ncclReduce (const void* sbuff, void* rbuff, size_t cnt, ncclDataType_t dtype, ncclRedOp_t op, int root, ncclComm_t comm, cudaStream_t s);
ncclResult_t ncclBroadcast (const void* sbuff, void* rbuff, size_t cnt, ncclDataType_t dtype, int root, ncclComm_t comm, cudaStream_t s);
ncclResult_t ncclAllReduce (const void* sbuff, void* rbuff, size_t cnt, ncclDataType_t dtype, ncclRedOp_t op, ncclComm_t comm, cudaStream_t s);
ncclResult_t ncclReduceScatter (const void* sbuff, void* rbuff, size_t rcnt, ncclDataType_t dtype, ncclRedOp_t op, ncclComm_t comm, cudaStream_t s);
ncclResult_t ncclAllGather (const void* sbuff, void* rbuff, size_t scnt, ncclDataType_t dtype, ncclComm_t comm, cudaStream_t s);

// Point to point communication - since 2.7
ncclResult_t ncclSend (const void* sbuff, size_t cnt, ncclDataType_t dtype, int peer, ncclComm_t comm, cudaStream_t s);
ncclResult_t ncclRecv (void* rbuff, size_t cnt, ncclDataType_t dtype, int peer, ncclComm_t comm, cudaStream_t s);

// Fuse operations together - since 2.0
ncclResult_t ncclGroupStart();
ncclResult_t ncclGroupEnd();

// Custom scaling factor - since 2.11
ncclResult_t ncclRedOpCreatePreMulSum(ncclRedOp_t *op, void *scalar, ncclDataType_t datatype, ncclScalarResidence_t residence, ncclComm_t comm);
ncclResult_t ncclRedOpDestroy(ncclRedOp_t op, ncclComm_t comm);

// Misc
ncclResult_t ncclGetVersion(int *version);
const char* ncclGetLastError(ncclComm_t comm);
ncclResult_t ncclCommCount(const ncclComm_t comm, int* count);
ncclResult_t ncclCommCuDevice(const ncclComm_t comm, int* device);
ncclResult_t ncclCommPeerRank(const ncclComm_t comm, int* rank);
```

Gen UniqueId



Init Bootstrap (00B)



Init Communicator

1. topology detection
2. graph search
3. graph connect



Launch CommOps

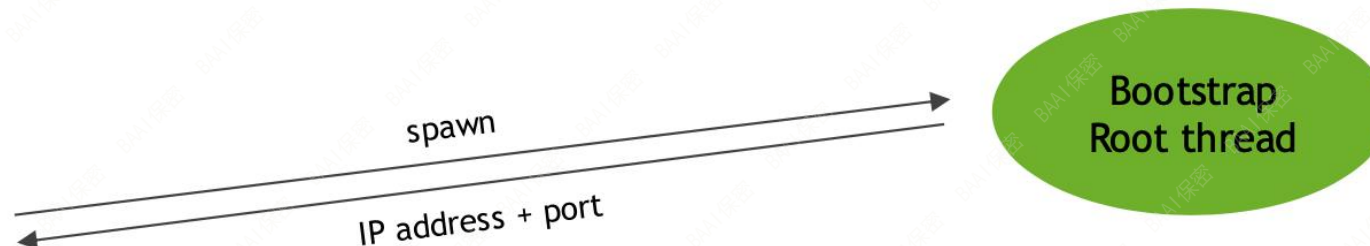


Destroy Communicator

NCCL workflow – UniqueId&Bootstrap

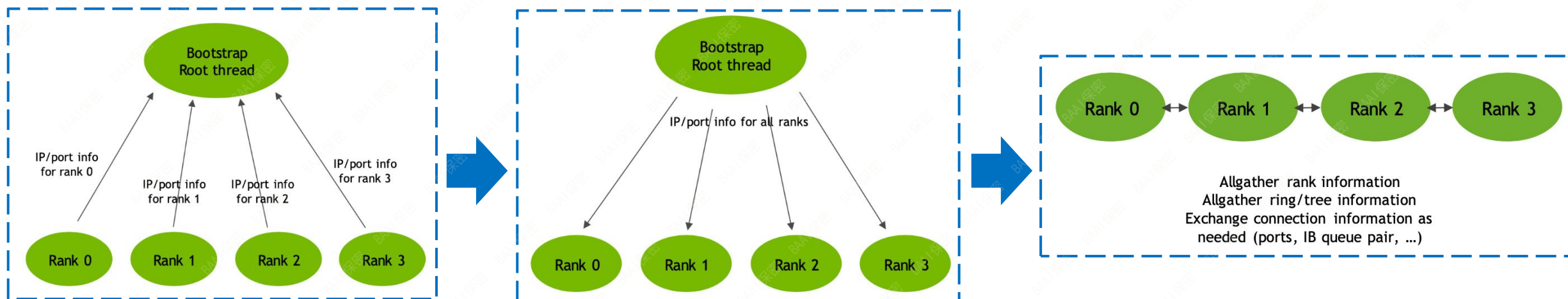
Once (typically on worker0):

```
ncclUniqueId id;  
ncclGetUniqueId(&id);  
broadcast_to_all_ranks(&id);
```



On all parallel workers:

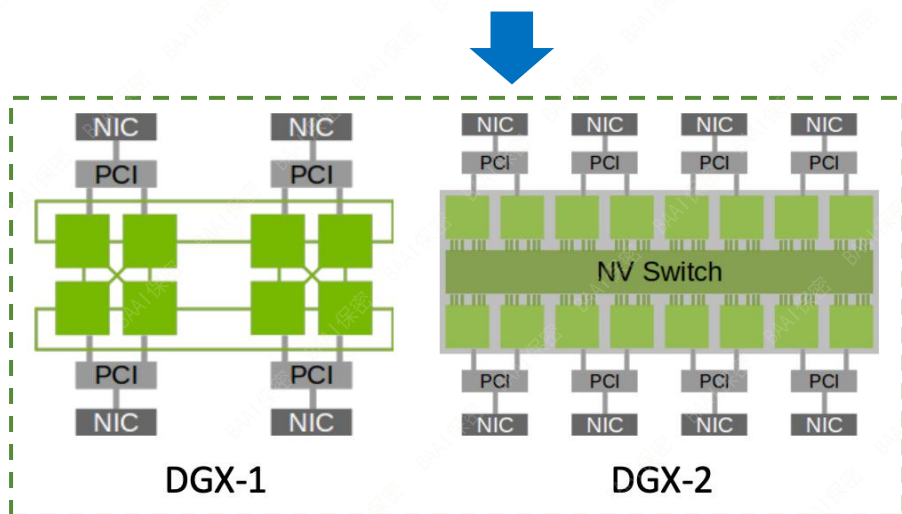
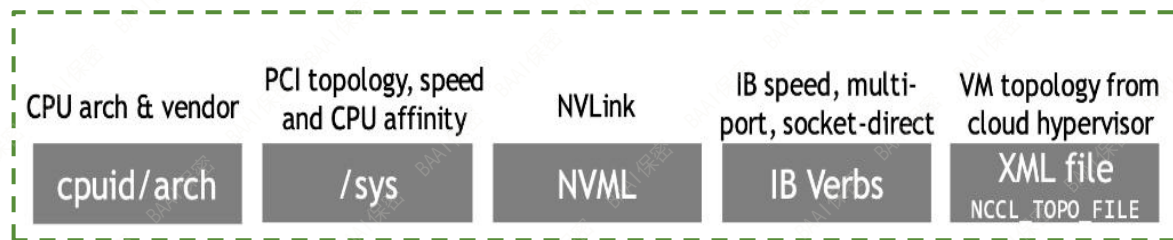
```
get_unique_id(&id);  
ncclCommInitRank(&comm, nranks, &id, rank);  
ncclAllReduce(..., comm);
```



NCCL工作流 – Topo&Graph

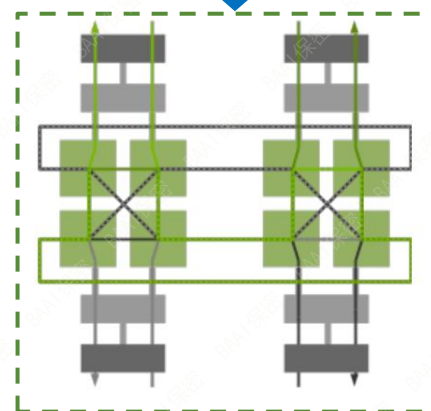
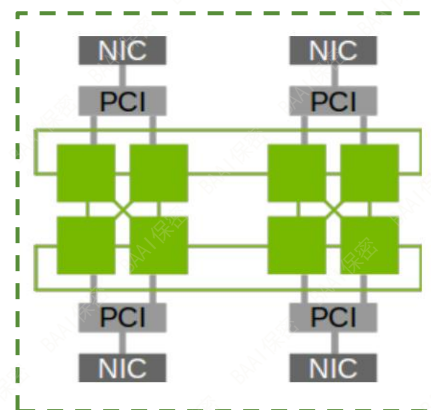
Topology detection

- ① 将GPU、NIC、CPU、PCI、NVLink、NVSwitch等的连接关系构建成图



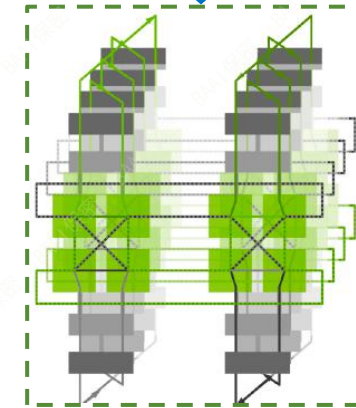
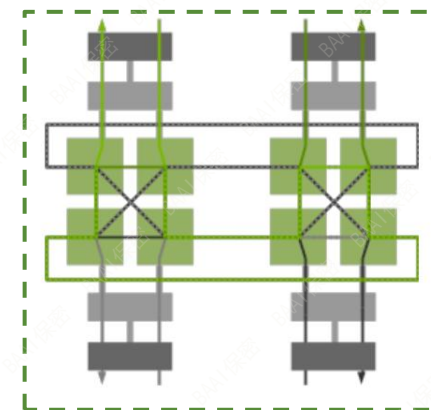
Graph search

- ① 节点内多路径搜索最大化 intra/inter-node 带宽
- ② 延迟带宽建模搜索最佳算法和协议



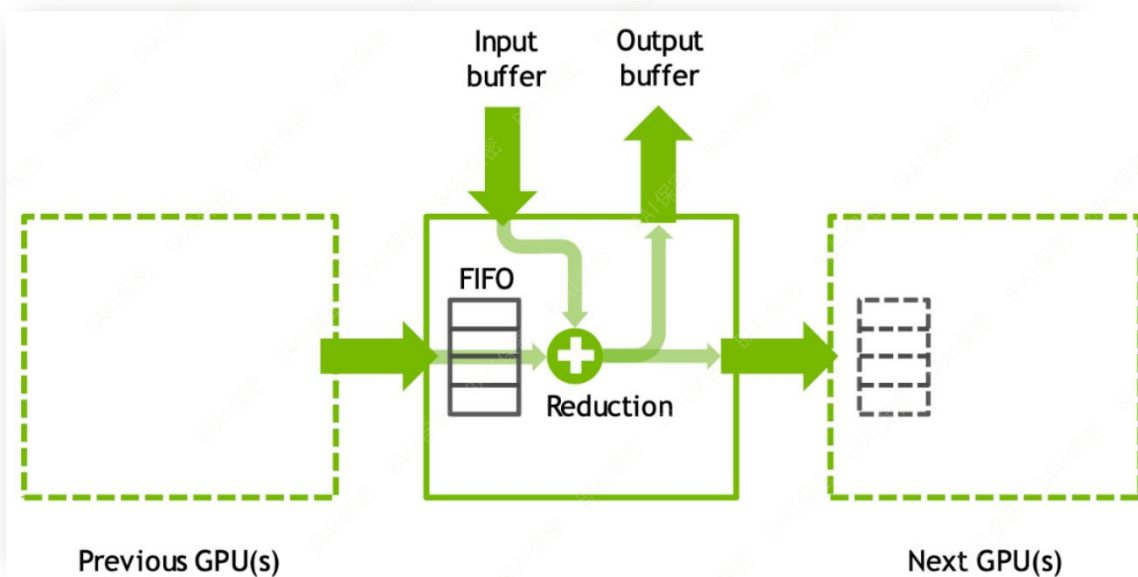
Graph connect

- ① 将不同节点graph连接起来
- ② 通过中间FIFO连接GPU



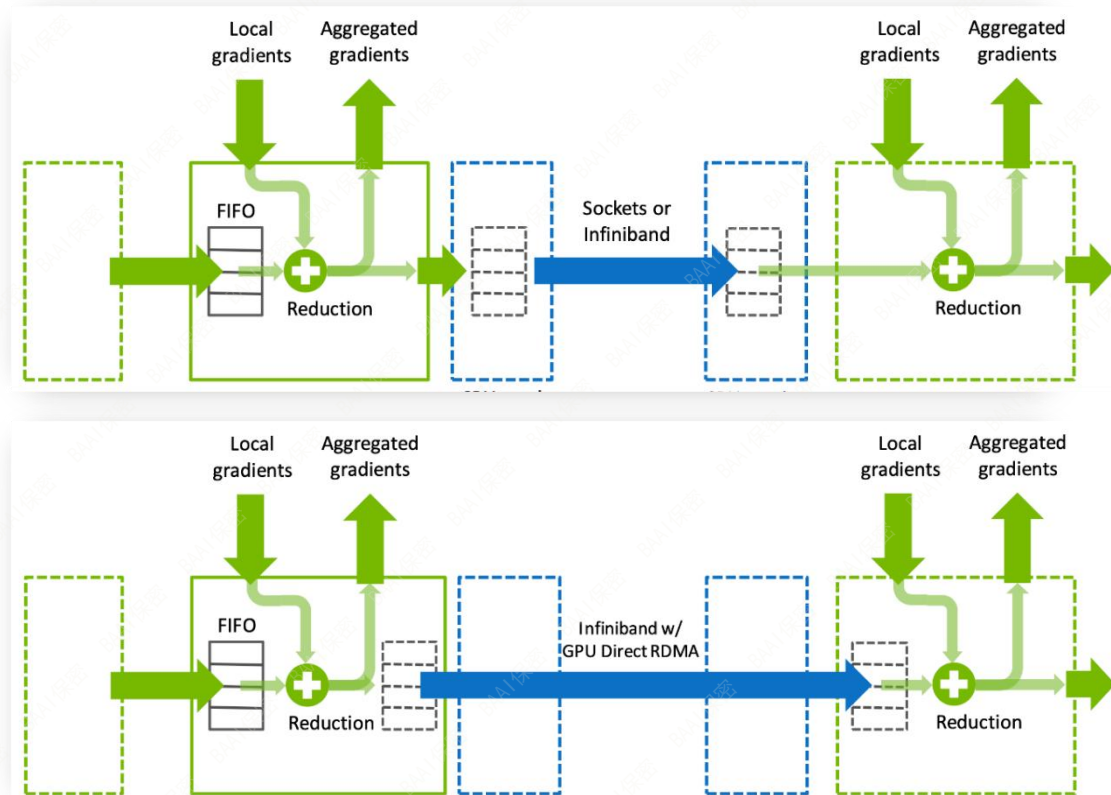
Intra-node Communication Op

- ① runs on the GPU
- ② receives and sends from other peers through internal FIFOs
- ③ perform reductions and copies with local and remote buffers



Inter-node Communication Op

- ① CPU-initiated network transfer
- ② TCP sockets, IB/RoCE, GDR



Algorithm

- ① Ring, Tree
- ② CollnetChain, CollnetDirect (SHARP)
- ③ NVLS, NVLSTree (NVLink SHARP)
- ④ PAT

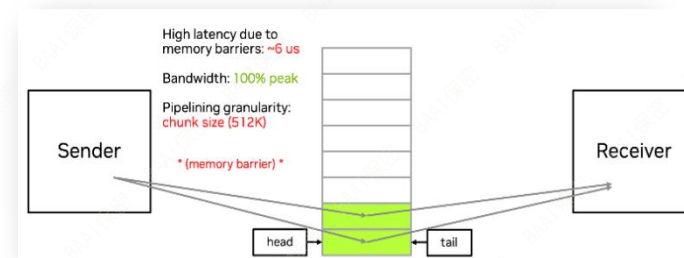
Supported algos for CommOps

- ① AllReduce: Ring, Tree, CollnetChain, CollnetDirect, NVLS, NVLSTree
- ② AllGather: Ring, CollnetDirect, NVLS, PAT
- ③ ReduceScatter: Ring, CollnetDirect, NVLS, PAT
- ④ Reduce: Ring
- ⑤ Broadcast: Ring
- ⑥ SendRecv: None

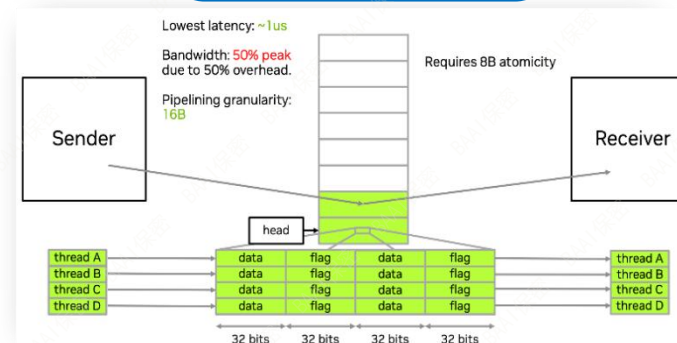
Supported algos for protos

- ① Simple: Ring, Tree, CollnetChain, CollnetDirect, NVLS, NVLSTree, PAT
- ② LL/LL128: Ring, Tree

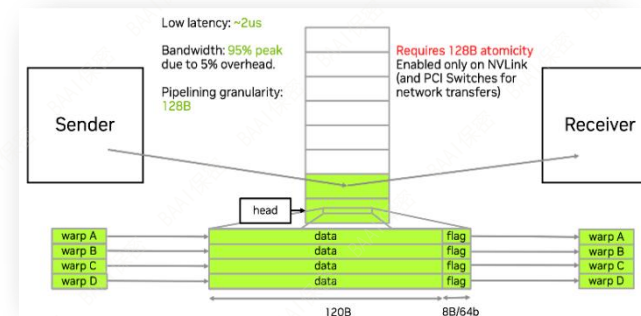
Protocol Simple

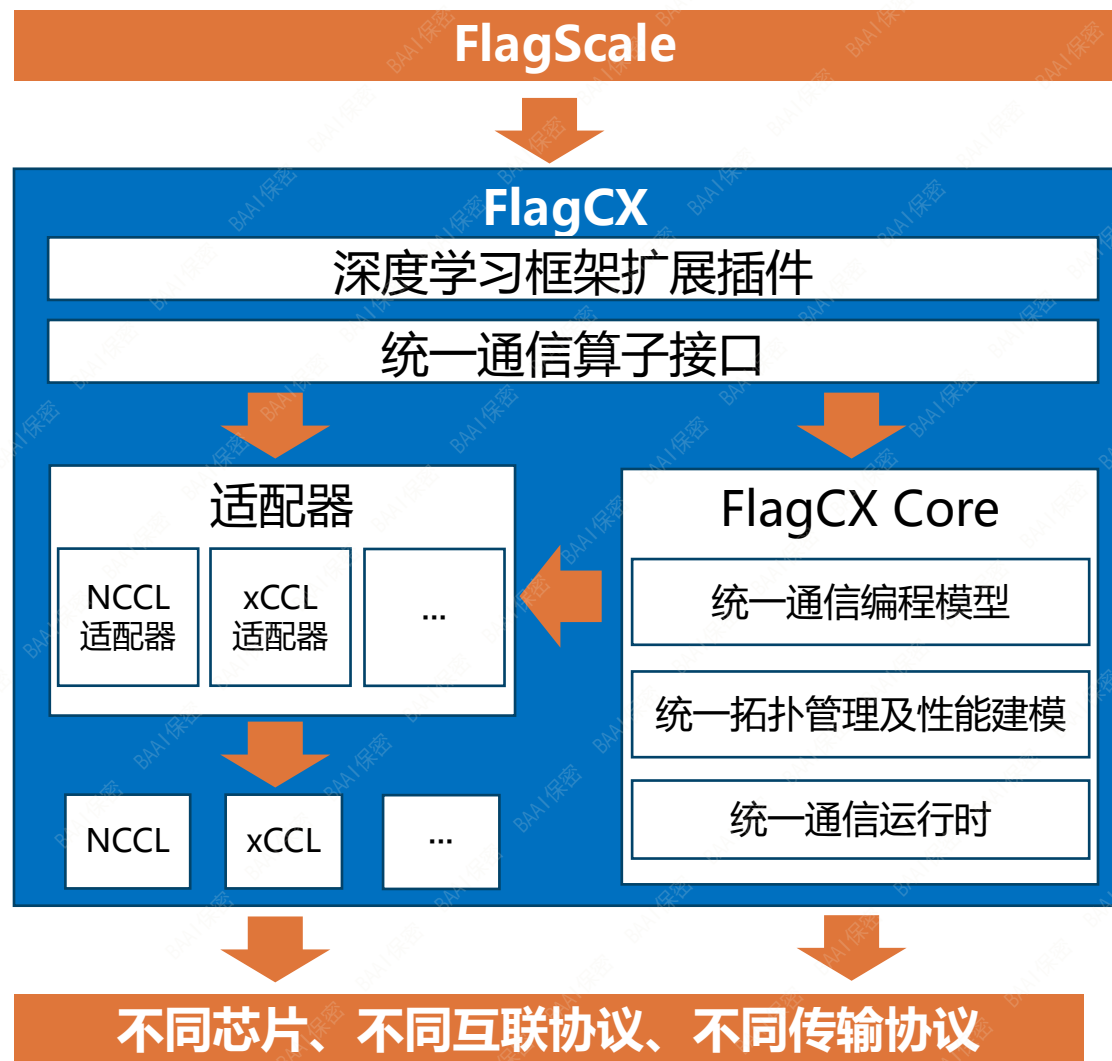


Protocol LL



Protocol LL128





开源地址: <https://github.com/FlagOpen/FlagCX>

设计目标

- › 实现同一芯片大规模高效稳定通信 (容错、故障检测)
- › 实现不同芯片高效通信, 支持跨芯高效异构混训
- › 实现基于统一编程模型的异构集合通信算法编译模拟

最新特性

- › 支持8+3种原生通信库和自动拓扑探测功能
- › 支持11种异构集合通信算法, 适用多网卡/单网卡等多种复杂场景
- › 提供FlagScale + FlagCX高效异构混训全栈开源方案; 支持百度飞桨+FlagCX跨芯通信; 原生集成浪潮元脑平台

当前已支持功能

通信操作支持

(all)reduce
(all)gather
(reduce)scatter
alltoall(v)
send/recv
broadcast

适配器支持

nvidia
iluvatar corex
cambricon
metax
kunlunxin
du
ascend
musa

框架支持

PyTorch
Paddle

拓扑支持

服务器拓扑感知
最优网卡匹配

三类抽象

- ① 通信库APIs
- ② 设备Runtime APIs
- ③ 传输APIs

CCLAdaptor

- * 通信子创建/销毁
- * 通信操作
- * 组操作
- * ...

DeviceAdaptor

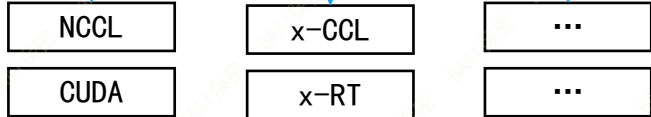
- * 内存分配/释放/拷贝
- * 流管理
- * 事件管理
- * ...

TransportAdaptor

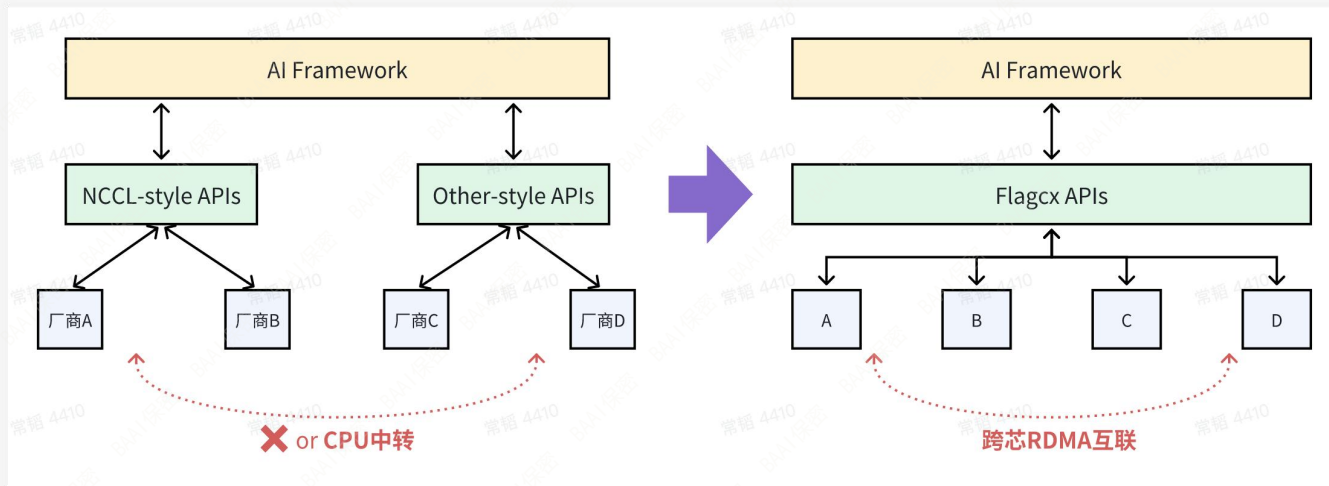
- * NET: IB/RoCE , TCP/IP
- * P2P: ...
- * SHM: ...



Unified Chip Interface (CCLAdaptor + DeviceAdaptor)



Unified Transport Interface (TransportAdaptor)



FlagCX API (NCCL-style API)

// Initialize, and destroy

```
flagcxResult_t flagcxHandleInit(flagcxHandlerGroup_t *handler);  
flagcxResult_t flagcxHandleFree(flagcxHandlerGroup_t handler);  
flagcxResult_t flagcxGetUniqueId(flagcxUniqueId_t *uniqueId);  
flagcxResult_t flagcxCommInitRank(flagcxComm_t *comm, int n ranks, flagcxUniqueId_t commId, int rank);  
flagcxResult_t flagcxCommDestroy(flagcxComm_t comm);
```

// Communication operations

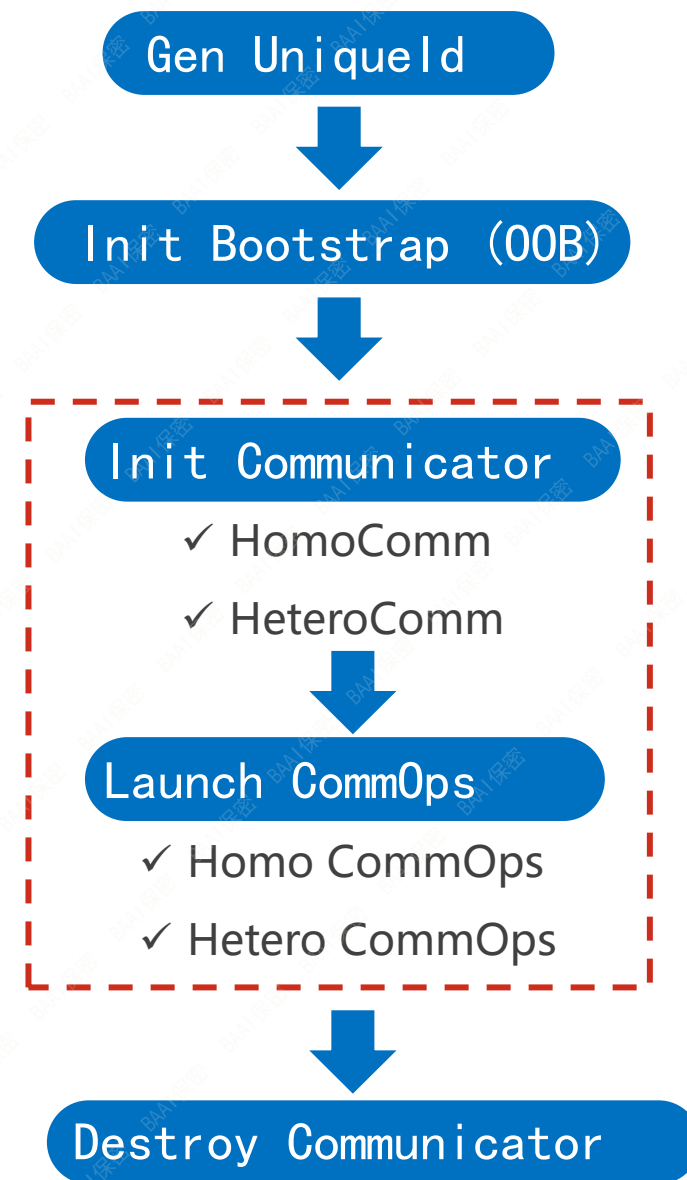
```
flagcxResult_t flagcxBarrier(flagcxComm_t comm, flagcxStream_t stream);  
flagcxResult_t flagcxReduce(const void *sendbuff, void *recvbuff, size_t count, flagcxDataType_t datatype,  
flagcxRedOp_t op, int root, flagcxComm_t comm, flagcxStream_t stream);  
flagcxResult_t flagcxBroadcast(const void *sendbuff, void *recvbuff, size_t count, flagcxDataType_t datatype, int root,  
flagcxComm_t comm, flagcxStream_t stream);  
flagcxResult_t flagcxAllReduce(const void *sendbuff, void *recvbuff, size_t count, flagcxDataType_t datatype,  
flagcxRedOp_t op, flagcxComm_t comm, flagcxStream_t stream);  
flagcxResult_t flagcxReduceScatter(const void *sendbuff, void *recvbuff, size_t recvcount, flagcxDataType_t datatype,  
flagcxRedOp_t op, flagcxComm_t comm, flagcxStream_t stream);  
flagcxResult_t flagcxAllGather(const void *sendbuff, void *recvbuff, size_t sendcount, flagcxDataType_t datatype,  
flagcxComm_t comm, flagcxStream_t stream);  
flagcxResult_t flagcxSend(const void *sendbuff, size_t count, flagcxDataType_t datatype, int peer, flagcxComm_t comm,  
flagcxStream_t stream);  
flagcxResult_t flagcxRecv(void *recvbuff, size_t count, flagcxDataType_t datatype, int peer, flagcxComm_t comm,  
flagcxStream_t stream);
```

// Group ops

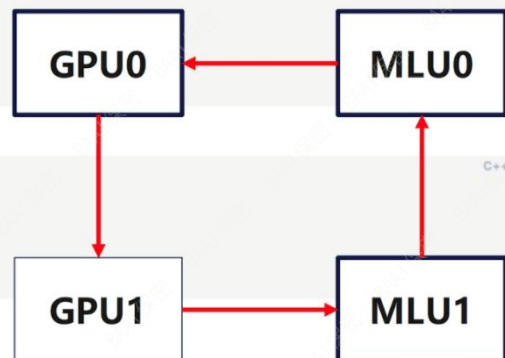
```
flagcxResult_t flagcxGroupStart(flagcxComm_t comm);  
flagcxResult_t flagcxGroupEnd(flagcxComm_t comm);
```

// Others

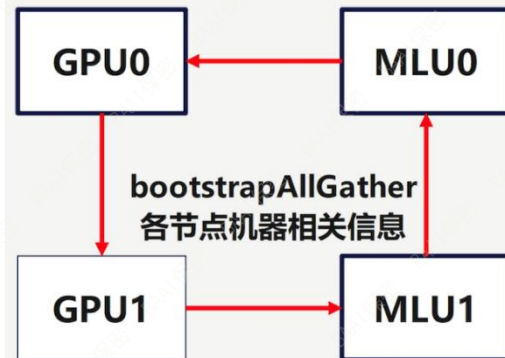
...



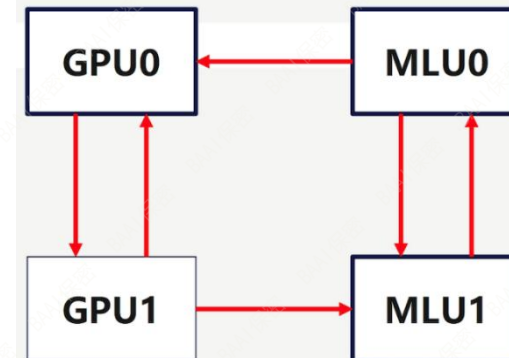
1. Gen uniqeld&Init bootstarp



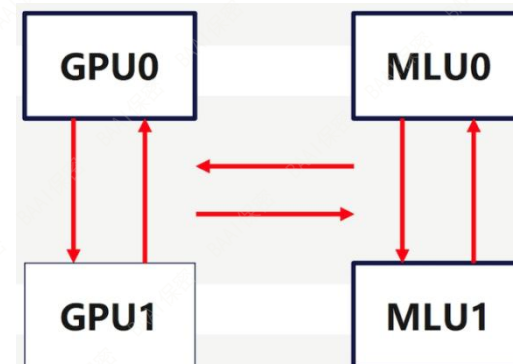
2. Collect cluster info



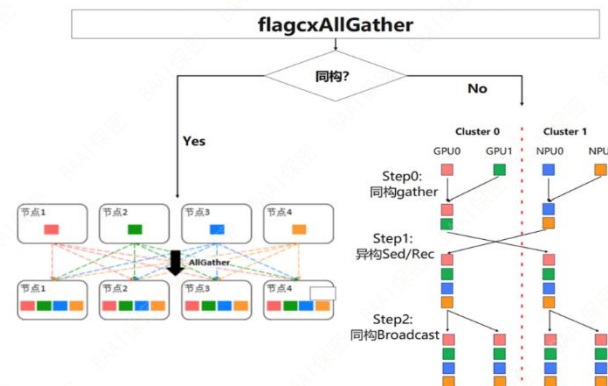
3. Init homo comm



4. Init hetero comm



5. Launch comm op



6. Destroy comm

几个关键点

1. 常见集合通信操作介绍
 - 规约: AllReduce, ReduceScatter, Reduce
 - 数据传输: Broadcast, Scatter, Gather, AllGather, AlltoAll
 - 同步: Barrier
2. AI通信范式
 - AllReduce/ReduceScatter/AllGather
 - AlltoAll/SendRecv
3. NCCL通信库的工作流
 - uniqueId generation&bootstrap initialization
 - topology detection&graph search&graph connect
 - commOp execution
4. FlagCX通信库的工作流
 - uniqueId generation&bootstrap initialization
 - homoComm&heteroComm initialization
 - commOp execution

欢迎各位同学贡献FlagOpen开源社区！

FlagCX: [*https://github.com/FlagOpen/FlagCX*](https://github.com/FlagOpen/FlagCX)

FlagScale: [*https://github.com/FlagOpen/FlagScale*](https://github.com/FlagOpen/FlagScale)



联系我们

邮件: job@baai.ac.cn

电话: 010 - 6893 3383

地址: 北京市海淀区成府路150号智源大厦



智源公众号